

TP Guidé — Jour 1

Préparation d'un corpus textuel en Python

Module NLP — Natural Language Processing — Bloc 2 · Durée : 3h30

Enseignant : Dr. Abdelhak TOUITI

Objectifs du TP

- Construire un corpus de travail et le charger avec **pandas**.
- Nettoyer un corpus textuel (bruit, doublons, formats hétérogènes).
- Normaliser, tokeniser et lemmatiser un texte en français avec **spaCy**.
- Retirer les mots vides tout en préservant le sens (négation).
- Documenter chaque étape du pipeline — ce document devient la base de la **Section 1 de votre dossier technique NLP**.

1. Mise en place de l'environnement (PowerShell)

Ce TP est rédigé pour un poste **Windows / PowerShell**. Ouvrez un terminal PowerShell (et non l'invite de commandes classique) dans le dossier où vous souhaitez travailler.

```
# 1. Creer et se deplacer dans le dossier de travail
mkdir TP_Jour1
cd TP_Jour1

# 2. Creer un environnement virtuel Python
python -m venv venv

# 3. Activer l'environnement virtuel (PowerShell)
.\venv\Scripts\Activate.ps1

# 4. Installer les librairies necessaires
pip install pandas spacy jupyter

# 5. Telecharger le modele francais de spaCy
python -m spacy download fr_core_news_sm
```

Remarque

Si l'étape 3 échoue avec un message du type « *l'exécution de scripts est désactivée sur ce système* », exécutez d'abord la commande suivante puis recommencez :

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

Tous les outils utilisés dans ce TP (**pandas**, **spaCy**, **Jupyter**) sont **gratuits et open source** — aucune installation payante n'est nécessaire.

Une fois l'environnement prêt, lancez Jupyter et créez un nouveau notebook `TP_Jour1.ipynb` :

```
jupyter notebook
```

2. Constitution du corpus de travail

Nous travaillons sur un corpus d'avis clients (fictif) concernant une plateforme d'e-learning. Copiez le code ci-dessous dans la première cellule de votre notebook : il crée directement le fichier `corpus_avis.csv` dans votre dossier de travail (aucun téléchargement nécessaire).

```
1 import pandas as pd
2
3 avis = [
4     ("Super plateforme, j'ai beaucoup appris !!! :)", 5),
5     ("TRES DECEVANT... le support ne repond jamais.", 1),
6     ("Bien mais un peu cher pour ce que c'est.", 3),
7     ("<p>Interface claire et agreable a utiliser.</p>", 4),
8     ("nul, ca plante tout le temps http://formaplust.example.com/bug",
9      1),
10    ("Je recommande vivement, contenu de qualite :)", 5),
11    ("Pas mal du tout, quelques bugs mineurs.", 4),
12    ("Service client injoignable, tres mecontent !!!", 1),
13    ("Excellent rapport qualite-prix, a essayer.", 5),
14    ("Bien mais un peu cher pour ce que c'est.", 3),
15    ("Les videos sont trop courtes a mon gout.", 3),
16    ("Je ne suis pas satisfait du tout, remboursez-moi.", 1),
17    ("Application fluide, aucun souci depuis 3 mois.", 5),
18    ("Moyen... l'ergonomie pourrait etre amelioree.", 2),
19    ("PARFAIT ! exactement ce qu'il me fallait <br> Merci", 5),
20    ("Corect sans plus, rien d'exceptionnel", 3),
21    ("Trop de publicites, ca devient agacant.", 2),
22    ("Une vraie pepite pour apprendre le NLP", 5),
23 ]
24
25 df = pd.DataFrame(avis, columns=["avis", "note"])
26 df.to_csv("corpus_avis.csv", index=False, encoding="utf-8")
27 print(df.shape)
```

✓ Point de contrôle

La commande `print(df.shape)` doit afficher (18, 2) : 18 avis, 2 colonnes (avis, note).

Remarque

Ce corpus contient volontairement des imperfections : balises HTML, une URL, des mots en majuscules, une faute d'orthographe, des émoticônes et une ligne **strictement dupliquée**. Vous allez les traiter dans les étapes suivantes.

3. Chargement et inspection du corpus

```
1 df = pd.read_csv("corpus_avis.csv")
2 print(df.shape)
3 df.head(10)
```

À faire : affichez également `df.info()` et repérez, en observant les lignes, au moins trois types de bruit présents dans le corpus (vous les documenterez à l'étape 8).

4. Nettoyage du corpus

On écrit une fonction de nettoyage réutilisable. Complétez les deux zones TODO.

```

1  import re
2
3  def clean_text(text):
4      text = text.lower()
5      # Supprimer les balises HTML, ex: <p>, <br>
6      text = re.sub(r"<.*?>", " ", text)
7      # TODO : supprimer les URLs (indice : commencent par http ou https)
8      text = re.sub(r"", " ", text)          # <-- completez la regex
9      # Supprimer la ponctuation excessive (!!!, ..., etc.)
10     text = re.sub(r"[!]{2,}", "!", text)
11     text = re.sub(r"[.]{2,}", ".", text)
12     # TODO : supprimer les espaces multiples et espaces en debut/fin
13     text = re.sub(r"", " ", text).strip()  # <-- completez la regex
14     return text
15
16 df["avis_clean"] = df["avis"].apply(clean_text)
17 df[["avis", "avis_clean"]].head(6)

```

Remarque

Indice pour la première regex : `r"http\S+"` supprime une URL et tout ce qui suit sans espace.
Indice pour la seconde : `r"\s+"` correspond à une ou plusieurs espaces.

Supprimons maintenant les doublons détectés à l'étape précédente :

```

1  avant = len(df)
2  df = df.drop_duplicates(subset="avis_clean").reset_index(drop=True)
3  print(f"Lignes avant : {avant} | après suppression des doublons : {len(df)}")

```

✓ Point de contrôle

Après suppression des doublons, le corpus doit contenir 17 lignes (un doublon exact a été supprimé).

5. Normalisation

À faire : en français, la suppression des accents fait perdre du sens (« la » vs « là »). Nous choisissons donc de **conserver les accents** et de ne normaliser que la casse et la ponctuation (déjà fait à l'étape précédente). Ce choix — et sa justification — doit apparaître dans votre documentation (étape 8).

6. Tokenisation et lemmatisation avec spaCy

```

1  import spacy
2
3  nlp = spacy.load("fr_core_news_sm")
4
5  def tokenize_lemmatize(text):
6      doc = nlp(text)

```

```

7     return [token.lemma_ for token in doc if token.is_alpha]
8
9 df["tokens"] = df["avis_clean"].apply(tokenize_lemmatize)
10 df[["avis_clean", "tokens"]].head(3)

```

✓ Point de contrôle

Pour la première ligne du corpus, la liste `tokens` doit contenir des formes lemmatisées telles que "apprendre" (et non "appris").

7. Suppression des mots vides (avec prudence)

```

1  # On recupere la liste de mots vides de spaCy, en retirant
   # volontairement
2  # la negation pour ne pas perdre le sens des avis negatifs
3  stop_words = nlp.Defaults.stop_words - {"ne", "pas", "non", "sans"}
4
5  def remove_stopwords(tokens):
6      return [t for t in tokens if t not in stop_words]
7
8  df["tokens_final"] = df["tokens"].apply(remove_stopwords)
9  df[["avis_clean", "tokens_final"]].head(3)

```

Remarque

C'est le point évoqué en cours ce matin : si on retire "pas" de « je ne suis **pas** satisfait », le sens de l'avis s'inverse. On exclut donc explicitement la négation de la liste des mots vides.

8. Statistiques rapides avant / après

```

1  nb_tokens_avant = df["avis_clean"].apply(lambda t: len(t.split())).sum()
2  nb_tokens_apres = df["tokens_final"].apply(len).sum()
3  vocabulaire = set(t for tokens in df["tokens_final"] for t in tokens)
4
5  print(f"Tokens avant nettoyage des mots vides : {nb_tokens_avant}")
6  print(f"Tokens apres nettoyage des mots vides : {nb_tokens_apres}")
7  print(f"Taille du vocabulaire final : {len(vocabulaire)}")

```

À faire : notez ces trois chiffres — ils ouvriront la séance d'analyse exploratoire (EDA) de demain matin.

9. Documenter votre pipeline

Complétez ce mini-rapport dans une cellule Markdown à la fin de votre notebook. Il constitue le cœur de la Section 1 de votre dossier technique.

Source du corpus :
Volume : avis avant nettoyage → après suppression des doublons
Bruit identifié :
Règles de nettoyage appliquées :
Choix de normalisation (accents, casse) et justification :
Traitement des mots vides et de la négation :
Taille du vocabulaire final :

10. Pour aller plus loin (optionnel)

- Remplacez le corpus fourni par un corpus de votre choix (export CSV, avis publics, etc.) et rejouez tout le pipeline.
- Comparez le vocabulaire obtenu avec et sans lemmatisation.
- Testez l'effet de la suppression des accents sur deux ou trois exemples : que perd-on exactement ?

Livrable attendu

À la fin de la séance, déposez votre notebook `TP_Jour1.ipynb` (code exécuté + cellule de documentation complétée). Il constitue la **Section 1 — Préparation des données** de votre dossier technique NLP, qui sera complété demain (EDA et représentation) puis le Jour 3 (modélisation et évaluation).